AgentOps 2.2 简历项目描述与讲稿

简历项目描述

AgentOps：企业知识与数据 Copilot 的可治理 Agent 系统

技术栈：Java 25、Spring Boot 3.5.11、Spring Security、MySQL、Redis、RabbitMQ、Elasticsearch、Python、FastAPI、LangGraph、LangChain、React、Vite、Gemini API

项目描述：将原有 Java 智能体 MVP 重构为 `Java control plane + Python LangGraph runtime + React workspace` 的可治理 Agent 系统。系统支持 `SINGLE_GRAPH / TEAM_GRAPH`、session/message 持久化、run/step/event trace、`agent_runtime_checkpoint`、checkpoint replay、MCP 工具平面、human-in-the-loop、skill trace、runtime-based Eval 与可视化 Dashboard。Python runtime 通过 LangGraph 执行 `intake / memory / supervisor / tool / review / approval / finalize` 状态图，其中 `supervisor_plan / evidence_reviewer / finalize` 由 Gemini 驱动，并新增 `Adaptive-RAG` 前置路由与 `Chain-of-Draft` planner 约束，以降低无效规划和简单请求的长链路开销。

简历亮点

- 设计并实现 `Java control plane + Python LangGraph runtime` 双层架构，保留 Java 的安全、持久化与工具治理能力，同时补齐 `LangGraph / LangChain / multi-step orchestration / replay / HITL` 关键词。
- 基于 `session -> run -> step -> event -> human_task -> checkpoint -> memory_fact` 模型沉淀完整 agent 轨迹，支持事件回放、审批恢复与 checkpoint replay。
- 将 `kb_search / doc_fetch / sql_query` 统一收敛到 Java MCP tool plane，并补充 `riskLevel / approvalPolicy / timeoutBudget / retryPolicy / auditRequired` 风控元数据。
- 在 runtime 中实现 `SINGLE_GRAPH / TEAM_GRAPH`、`review / replan / bounded failure / human-in-the-loop / skill trace`，覆盖应用岗和平台岗常见追问。
- 在 `intake + supervisor` 链路加入 `Adaptive-RAG` 复杂度分流和 `Chain-of-Draft` planner 约束，让简单请求绕过冗余 planner、多步问题保留完整 replan 主链，同时降低规划 token 和延迟。
- 构建 React 工作台，集中展示聊天记录、graph 视图、step trace、SSE live events、审批面板和 Eval dashboard，提升项目演示冲击力。
- 升级 Eval 体系，直接执行真实 runtime 图，支持 `route / grounding / citation / approval / latency / retry / skills / orchestrationMode` 指标，形成 agent 质量闭环。

1 分钟讲稿

我把原来的 Java 智能体项目升级成了一个三层结构的 Agent 系统。Java 负责 control plane，也就是登录鉴权、session/message、run/step/event/checkpoint 持久化、MCP 工具平面、人工审批和评测；Python 用 FastAPI + LangGraph + LangChain 实现真正的 runtime，负责多步状态图执行、checkpoint replay、Adaptive-RAG 前置路由和 skill trace；前端则做成了一个工作台，能看聊天、graph trace、live events、审批和 Eval dashboard。这样项目就不再是“单轮 prompt + tool call”，而是一个真正具备 `multi-step orchestration、human-in-the-loop、replay、trace & eval` 的可治理 Agent 应用项目。

3 分钟讲稿

AgentOps 2.2 的核心改造是在 2.1 三层结构上继续补齐 runtime 硬度。现在 Java 不只是控制面，还承接 `agent_runtime_checkpoint + agent_run_event`，因此 run、step、approval、checkpoint、event 都在一个一致的系统记录源里。真正的多步执行交给 Python runtime，它用 LangGraph 组织 `intake_guardrail -> load_memory -> supervisor_plan -> knowledge_researcher / data_analyst -> evidence_reviewer -> human_approval -> finalize` 这条图，而且支持 `SINGLE_GRAPH / TEAM_GRAPH` 两种编排模式，`supervisor_plan / evidence_reviewer / finalize` 三个节点都由 Gemini 驱动。为了减少简单请求白跑 planner，我又在 intake 和 supervisor 之间补了一层 `Adaptive-RAG` 路由，把请求分成保守直答、单跳知识、单跳数据和多步问题；同时给 planner prompt 加了 `Chain-of-Draft` 约束，尽量用极简草稿完成内部规划，不输出冗长推理。

这样做有三个收益。第一，技术栈上真正补齐了大厂 JD 常出现的 `LangGraph / LangChain / MCP / human-in-the-loop`。第二，工程结构更合理，Java 依然是系统记录源，所有 session、run、step、approval、memory、eval 都在一个稳定的 control plane 里。第三，项目可演示性更强，前端工作台可以直接展示 graph 视图、step trace 和审批动作，不需要只靠 Swagger 讲故事。

工具层我保留了 `kb_search`、`doc_fetch`、`sql_query` 三类能力，但把它们统一为 Java MCP tool plane 暴露的工具，并补了 `riskLevel / approvalPolicy / timeoutBudget / retryPolicy / auditRequired` 这些治理字段。Python runtime 不直接读数据库，而是通过 LangChain tools 调 Java MCP，这样边界清晰，也更接近企业里的工具平面设计。为了让结果更像真正的 agent，我在 reviewer 节点里加入了 replan 和 bounded failure 逻辑，在高风险 SQL 场景里加入人工审批，并通过 `resumeToken + checkpointVersion` 做审批恢复校验，同时补了一条从最新 checkpoint 做 replay / continue 的链路。最后 Eval 不再走旧的 Java 旁路链，而是直接执行真实 runtime 图，把 route、grounding、citation、approval、latency、skills 等指标串起来，形成完整质量闭环。